

# Technical Stack & Architecture Specification

**Project:** Tech Job Aggregator & Analytics Hub **Version:** 1.0.0

## 1. System Architecture Overview

The platform utilizes a **Minimal Microservices Architecture** deployed via Docker Compose. This cleanly separates the data-ingestion workloads from the user-facing application, ensuring UI snappiness remains entirely unaffected by scraping spikes or target-site timeouts.

The architecture comprises four core pillars:

1. **Frontend Client:** Handles rendering, filtering, and data visualization.
2. **Core API:** Manages authentication, business logic, subscriptions, and database queries.
3. **Aggregation Pipeline (Scraper):** Autonomous workers executing cron jobs, parsing HTML/APIs, and normalizing payloads.
4. **Data Layer:** Relational storage paired with an in-memory datastore for caching and job queues.

## 2. Technical Stack Definition

### 2.1. Frontend Client (UI/UX)

- **Framework:** Next.js (React) using the App Router. Delivers superior SEO for job listings and aggressive server-side caching.
- **Styling:** Tailwind CSS. Guarantees rapid layout construction and fluid responsive design without CSS bloat.
- **State & Data Fetching:** React Query (TanStack Query). Manages client-side caching, background refetching, and pagination state for the job feed.
- **Data Visualization:** Recharts or Chart.js. Renders the market trendline graphs and bar charts required for the Analytics Engine.

### 2.2. Core API (The Brain)

- **Runtime & Framework:** Node.js with NestJS (TypeScript). Enforces strict architectural boundaries, dependency injection, and modularity—ideal for an "elite portfolio piece."
- **ORM:** Prisma. Generates fully typed database clients, ensuring schema consistency between the Core API and the database.

- **Authentication:** NextAuth.js or JWT-based Passport strategy within NestJS. Secures the Pro Plan routes and protects user webhook configurations.

## 2.3. Aggregation Pipeline (Scraper Microservice)

- **Runtime:** Node.js (TypeScript) or Python. (Node.js keeps the entire monorepo in a unified language).
- **Extraction Tools:** Cheerio (for lightning-fast static HTML parsing) and Puppeteer (for bypassing JavaScript-rendered job boards).
- **Job Queue Engine:** BullMQ (backed by Redis). Manages scraping concurrency, handles automatic retries on connection failures, and respects target-site rate limits to prevent IP bans.
- **Scheduling:** Node-cron or BullMQ's native repeatable jobs.

## 2.4. Data & Caching Layer

- **Primary Database:** PostgreSQL. Provides robust relational integrity, crucial for linking Jobs to Tags, Companies, and User Alerts.
- **In-Memory Cache & Queue:** Redis. Serves a dual purpose: caching complex aggregate queries for the Analytics Engine and acting as the message broker for BullMQ.

# 3. Database Architecture & Indexing Strategy

To guarantee sub-100ms response times as historical job records scale into the millions, the PostgreSQL schema requires strict indexing.

- **B-Tree Indexes:** Apply to `posted_at`, `company_id`, and `experience_level`.
- **GIN Indexes (Generalized Inverted Index):** Apply to the normalized `description` and `title` fields to power ultra-fast, full-text keyword searches.
- **Unique Constraints (Deduplication):** Implement a composite unique key combining `(company_name, normalized_title, date_posted)` to reject duplicate entries at the database level during the scraper's ingestion phase.
- **Soft Deletions:** Job records use a `status` boolean (Active/Expired) rather than destructive deletion. This preserves the historical dataset required by the Pro Plan's Market Share Shift analytics.

# 4. Third-Party Integrations

- **Instant Alert Engine:** Telegram Bot API. Webhooks push formatted job payloads directly to Pro users.
- **Email Digest System:** Nodemailer integrated with SendGrid or AWS SES to dispatch batched HTML digests reliably.

## 5. Infrastructure & Deployment

The stack targets a standard Linux VPS environment, optimized for modest hardware.

- **Containerization:** Docker & Docker Compose. Every service (Frontend, API, Scraper, Postgres, Redis) runs in an isolated container.
- **Reverse Proxy / API Gateway:** Nginx or Traefik. Handles SSL termination (Let's Encrypt), routes traffic to the correct container, and provides basic DDOS protection.
- **CI/CD Pipeline:** GitHub Actions. Automates testing, linting, and deployment. On push to `main`, the pipeline builds the Docker images and triggers a zero-downtime deployment on the production server.
- **Monitoring (Optional but recommended):** Winston/Morgan for structured logging, heavily crucial for diagnosing scraper failures without SSH-ing into the server.